

GLOBALMART RETAIL DATA PLATFORM

Proof of Concept — Complete Project Outline

6 Phases • 14 Snowflake Features • 3 Source Files • 225,160 Rows

Covers: COPY INTO • JSON FLATTEN • Snowpipe • Streams •

1. Executive Summary

GlobalMart is a growing retail chain with 10 stores across Istanbul, Izmir, Ankara, and Antalya. The business generates data from three completely separate systems — a POS system, an ERP system, and IoT sensors — each producing a different file format, at a different frequency, owned by a different team.

This Proof of Concept builds a complete end-to-end data platform on Snowflake that unifies all three sources, demonstrates every major Snowflake engineering feature, and produces business-grade analytics queries that answer real operational questions.

Dimension	Detail
Source files	11 files across 3 formats — CSV, Parquet, JSON
Total rows	225,160 rows — 120K POS, 45K ERP, 60K IoT events
Snowflake features	14 features across 6 phases
Schemas	5 schemas: integrations · raw · staging · marts · utilities
Business output	Gross margin per store, IoT alert impact on sales, 3-way cross-file JOIN
Duration	4–6 hours end-to-end from file generation to business queries

2. Problem Statement

2.1 The Data Silos

None of the three source systems talk to each other. A store manager cannot answer: "Did the IoT temperature breach in cold storage cause lower food & beverage sales this week?" The answer lives across three disconnected files in three different formats.

System	File Format	Frequency	Volume	Business Question
POS (Point of Sale)	CSV — flat tabular	Daily per store	120,000 rows/year	What did we sell, to whom, at what price?
ERP (Orders & Inventory)	Parquet — columnar	Weekly bulk export	45,000 orders	What did we order, at what cost, on time?
IoT Sensors	JSON — nested arrays	Every 5 min/device	60,000 events/batch	What is physically happening in our stores?

2.2 Business Problems This POC Solves

- Revenue vs cost gap — POS shows revenue but ERP shows procurement cost. No single table connects them, so gross margin is invisible.
- Operational blindness — IoT temperature breach alerts are not connected to the sales drop they cause. The connection only exists after building this pipeline.
- No data recovery — when an analyst runs a bad UPDATE at 11 PM, there is no recovery mechanism. Time Travel solves this.
- No change tracking — when an ERP order status changes, nothing captures that change for downstream systems. Streams and Tasks solve this.
- Stale data — finance runs reports from yesterday's flat file. By the time it is reviewed, today's stock decisions are already wrong.

3. Project Architecture

The platform follows the Medallion Architecture — the industry standard for cloud data warehouses. Three layers separate concerns clearly: raw data preservation, business logic transformation, and analytical consumption.

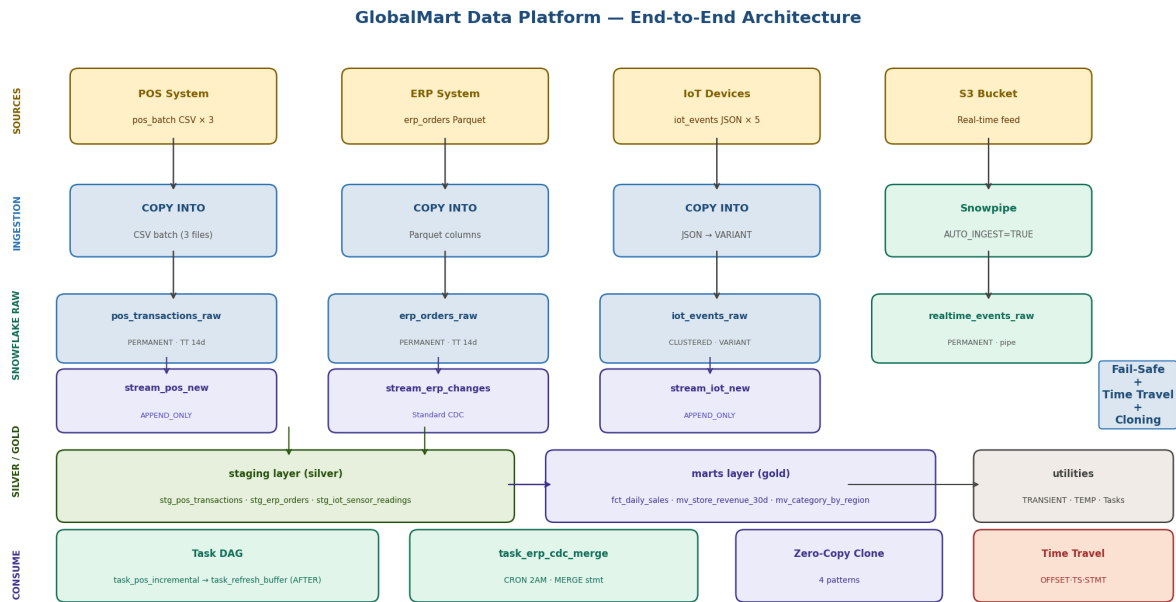


Figure 1: End-to-end GlobalMart data platform — from source files through to business query layer.

3.1 The Five Schemas

Schema	Layer	Purpose	Table Types
integrations	Infrastru cture	Storage integration, file formats, external stages. Never drop this schema.	Stages, File Formats
raw	Bronze	Exact copy of source files. Never modified. Source of truth for Time Travel recovery.	PERMANENT — 14d retention
staging	Silver	Cleaned, typed, deduplicated. Business logic applied. Fed by Streams and Tasks.	PERMANENT — 7d retention
marts	Gold	Business-facing aggregates, fact tables, Materialized Views. What dashboards query.	PERMANENT — 30d retention
utilities	Utilities	Transient staging buffers rebuilt nightly, temporary dedup tables, task definitions.	TRANSIENT + TEMPORARY

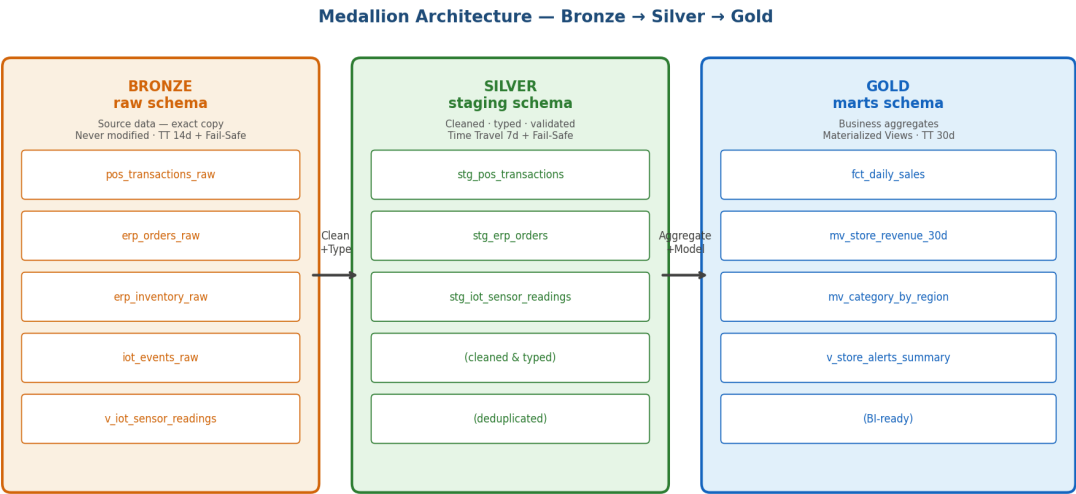


Figure 2: Medallion Architecture — Bronze, Silver, and Gold layers with table types and data flow.

4. Source Files — Role and Schema

Each file has a deliberate design. The columns, structure, and batching decisions were made to enable specific Snowflake features, not just to have data. Understanding each file's role is essential before writing any SQL.

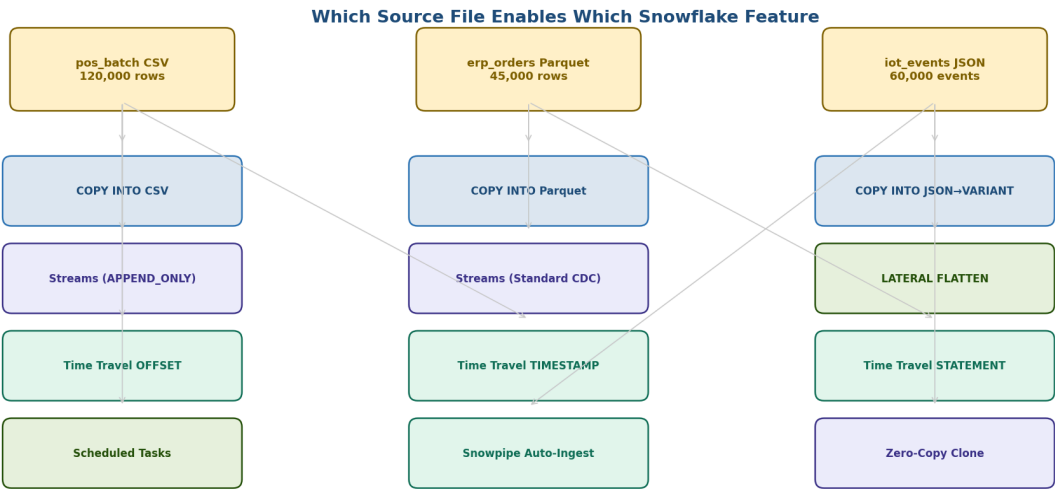


Figure 3: Which source file enables which Snowflake feature. Each file covers distinct features.

4.1 File 1 — pos_batch CSV files (120,000 rows total)

Role in the project
Primary revenue file. Drives: COPY INTO CSV, Streams (APPEND_ONLY), Time Travel (all 3 methods),
Task DAG (incremental load + child task chain), Materialized Views (source for fct_daily_sales),
Zero-copy clone (fct_daily_sales is the clone target), and the gross margin JOIN with ERP.
Split into 3 batches (Jan-Apr, May-Aug, Sep-Dec) so you can load them one at a time to trigger the Stream demo — loading all at once would prevent the incremental CDC demonstration.

Column	Type	Key Role in Project
transaction_id	VARCHAR (30)	Primary key — uniqueness check, Time Travel STATEMENT recovery target
store_id	VARCHAR (10)	JOIN key to ERP and IoT — STR_001 through STR_010 in all 3 files
category	VARCHAR (50)	JOIN key to ERP on category for gross margin calculation

Column	Type	Key Role in Project
transaction_date + time	DATE + VARCHAR	Combined into TIMESTAMP_NTZ in staging — drives all time-based queries
total_amount	FLOAT	Revenue metric — used in fct_daily_sales and Materialized Views
discount_pct	FLOAT	Target of the deliberate bad UPDATE in Time Travel STATEMENT demo
_source_file	VARCHAR	Auto-populated via METADATA\$FILENAME — file lineage tracking

4.2 File 2 — erp_orders.parquet (45,000 rows) + erp_inventory.parquet (160 rows)

Role in the project
Orders file drives: COPY INTO Parquet (\$1:field_name notation), Standard Stream (CDC — order_status updates), MERGE Task (ERP CDC merge nightly at 2AM), and the gross margin JOIN with POS.
Inventory file (160 rows) is used specifically for the UNDROP demo — dropped and instantly restored.
IMPORTANT: ERP product_sku values differ from POS by design. In real retail, POS scans a barcode while ERP uses an internal procurement SKU. Join on store_id + category, NOT product_sku.

Column	Key Detail
order_id	Primary key — unique per purchase order to a supplier
store_id	JOIN key — STR_001 through STR_010, identical to POS and IoT
order_status	Values: Pending/Confirmed/Shipped/Delivered/Delayed/Cancelled — UPDATED to trigger CDC stream
category	Matches POS categories exactly — the join key for gross margin analysis
total_cost	Procurement cost — compared against POS revenue for gross margin calculation
is_late	Boolean TRUE when actual_delivery > expected_delivery — ~70% in generated data
warehouse_id	5 warehouses: WH_IST_01/02, WH_ANK_01, WH_IZM_01, WH_ANT_01

4.3 File 3 — iot_events_batch_01..05.json (60,000 events total)

Role in the project
The ONLY file that enables LATERAL FLATTEN. The nested readings[] array (2 sensors per event) and alerts[] array (on ~5% of events) are specifically designed for the flatten demonstration.
Batches 01+02 load via manual COPY INTO for the initial flatten demo.
Batches 03+04+05 load via Snowpipe (pipe_iot_events) to demonstrate AUTO_INGEST.
store_id matches POS and ERP exactly — enabling the 3-way JOIN across all 3 sources.

JSON Field	Type	Access Syntax	Notes
event_id, store_id, timestamp	Top-level string	\$1:event_id::VARCHAR	Single colon — one level deep
metadata.firmware	Nested object	\$1:metadata:firmware::VARCHAR	Double colon path — two levels deep
metadata.battery_pct	Nested integer	\$1:metadata:battery_pct::INTEGER	Cast to INTEGER after extraction
readings[]	Array of objects	LATERAL FLATTEN(input => raw_payload:readings)	Explodes 1 row → 2 rows (one per sensor)
alerts[]	Array of objects	LATERAL FLATTEN(input => raw_payload:alerts)	Only 556/12000 events have alerts
r.value:sensor	Field inside array element	r.value:sensor::VARCHAR	r is the FLATTEN alias — each element is r.value

5. Six-Phase Project Outline

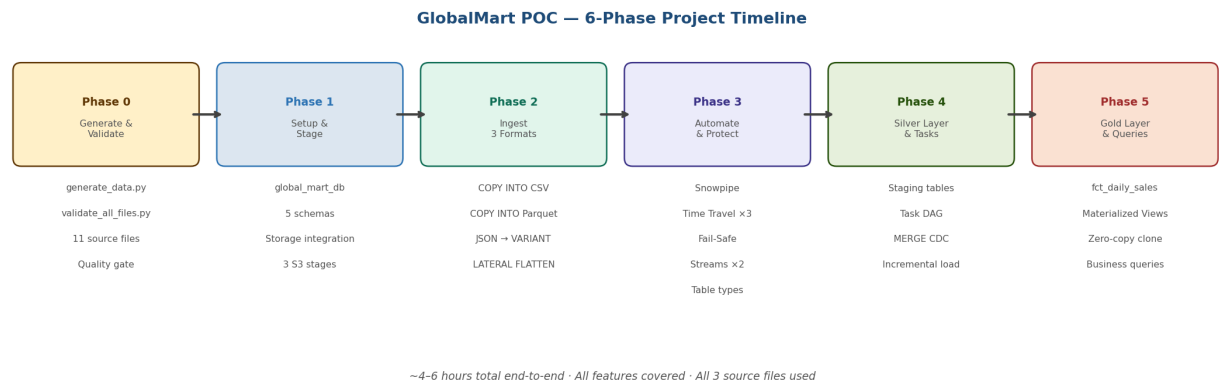


Figure 4: GlobalMart POC — 6-phase project timeline showing features, duration, and sequence.

Phase 0 — Generate and Validate Source Files

Run on your laptop before opening Snowflake. No Snowflake account needed.

Step	Action	Expected Output
0.1	pip install pandas pyarrow	Dependencies installed
0.2	python3 generate_data.py	/data/ folder with 11 files created in under 60 seconds
0.3	python3 validate_all_files.py	31 PASS, 1 WARN, 0 FAIL — all files schema-valid and cross-file JOINS feasible

Phase 1 — Snowflake Setup, Schemas, Warehouses, Stages

SQL Sections 1–3. One-time bootstrap. The storage integration is the only step requiring ACCOUNTADMIN.

Object Created	Name	Purpose
Database	global_mart_db	Root database for the entire project
Schema ×5	integrations, raw, staging, marts, utilities	One schema per concern — never mix objects between them
Warehouse ×2	gm_ingest_wh (XS), gm_transform_wh (S)	Separate compute for ingestion vs transformation
Storage integration	s3_globalmart_int	IAM role trust — no credentials stored. Required for Snowpipe.
File format ×3	ff_csv, ff_parquet, ff_json	One format per source type. ff_json must have STRIP_OUTER_ARRAY=TRUE

Object Created	Name	Purpose
External stage ×3	stg_pos, stg_erp, stg_iot	Each points to a separate S3 prefix. Separate stages = separate file formats.

Phase 2 — Ingest All 3 File Formats into Raw Tables

SQL Sections 4–5. Load ONLY pos_batch_jan_apr.csv (not all 3 batches). Hold May-Aug and Sep-Dec for the Stream demo in Phase 3. Load IoT batches 01+02 only. Reserve 03-05 for Snowpipe.

Feature	File Used	Table Target	Key SQL Pattern	Verify
COPY INTO CSV	pos_batch_jan_apr.csv	pos_transactions_raw	SELECT \$1,\$2...\$19 + METADATA\$FILENAME	COUNT(*) = ~39,521
COPY INTO Parquet	erp_orders.parquet	erp_orders_raw	SELECT \$1:order_id::VARCHAR, \$1:order_date::TIMESTAMP_NTZ...	COUNT(*) = 45,000
COPY INTO JSON→VARIANT	iot_events_batch_01+02	iot_events_raw	\$1 alone → VARIANT column, \$1:event_id for key fields	COUNT(*) = 24,000
LATERAL FLATTEN	iot_events_raw VARIANT	v_iot_sensor_readings (view)	LATERAL FLATTEN(input => raw_payload:readings)	COUNT = 48,000 (2×)
LATERAL FLATTEN (alerts)	iot_events_raw VARIANT	v_iot_alerts (view)	LATERAL FLATTEN(input => raw_payload:alerts) WHERE ARRAY_SIZE > 0	COUNT = ~1,112

Phase 3 — Automate and Protect

SQL Sections 6–10. The most feature-dense phase. Covers Snowpipe, all 3 Time Travel methods, Fail-Safe analysis, both Stream types, and table types.

Feature	File / Object	How Triggered	Key Verification
Snowpipe AUTO_INGEST	iot_events_batch_03+04+05	Pipe monitors stg_iot stage; files trigger automatically	Load batch_03 twice → second run STATUS=SKIPPED (idempotency)
TRANSIENT table	daily_sales_buffer	CREATE OR REPLACE TRANSIENT TABLE	INFORMATION_SCHEMA: FAILSAFE_BYTES = 0
TEMPORARY table	temp_iot_dedup	CREATE OR REPLACE TEMPORARY TABLE	Disappears after session ends — no SHOW TABLES entry
Time Travel OFFSET	pos_transactions_raw	Load May-Aug batch, then AT(OFFSET => -300)	Row count before load ≠ row count after load
Time Travel TIMESTAMP	pos_transactions_raw	AT(TIMESTAMP => \$load_ts) from COPY_HISTORY	Revenue query shows only Jan-Apr data

Feature	File / Object	How Triggered	Key Verification
Time Travel STATEMENT	pos_transactions_raw	Bad UPDATE → QUERY_HISTORY → BEFORE(STATEMENT)	discount_pct restored from -99 to original values
UNDROP TABLE	erp_inventory_raw	DROP TABLE then UNDROP TABLE	COUNT(*) = 160 rows restored instantly
Fail-Safe analysis	All permanent tables	TABLE_STORAGE_METRICS query	FAILSAFE_BYTES > 0 (permanent) vs = 0 (transient)
Stream APPEND_ONLY	pos_transactions_raw	Load Sep-Dec CSV batch	SYSTEM\$STREAM_HAS_DATA = TRUE after load
Stream Standard CDC	erp_orders_raw	UPDATE order_status Shipped→Delivered	METADATA\$ISUPDATE = TRUE in stream preview

Phase 4 — Silver Layer and Task DAG

SQL Sections 11–12. Full initial load into staging tables with cleaning. Task DAG activated — remember to resume child task before root task.

Object	Type	Trigger	Action	Verify
stg_pos_transactions	PERMANENT table	Initial full INSERT from raw	Cleans: INITCAP category, COALESCE nulls, UPPER payment, combined timestamp	COUNT = 120,000
stg_erp_orders	PERMANENT table	Initial full INSERT from raw	Copies all 45,000 orders with correct types	COUNT = 45,000
task_pos_incremental	ROOT task	WHEN stream_pos_new HAS_DATA, every 15 min	INSERT new POS rows from stream into stg_pos_transactions	TASK_HISTORY = SUCCEEDED
task_refresh_buffer	CHILD task	AFTER task_pos_incremental	INSERT OVERWRITE daily_sales_buffer (transient rebuild)	Runs after root, buffer refreshed
task_erp_cdc_merge	INDEPENDENT task	CRON 0 2 * * * UTC, WHEN stream_erp HAS_DATA	MERGE ON order_id — UPDATE status, INSERT new orders	MERGE rows updated in stg_erp_orders

Phase 5 — Gold Layer, Cloning, and Business Queries

SQL Sections 13–15. Build the business-facing layer and run the cross-file queries that are the reason the platform was built.

Feature	Object	How Created	Business Value
Fact table	fct_daily_sales	Aggregate stg_pos by date + store + category	Daily revenue, units, basket size, customer count per store

Feature	Object	How Created	Business Value
Materialized View	mv_store_revenue_30d	CREATE MATERIALIZED VIEW on fct_daily_sales	Pre-computed 30-day rolling revenue — dashboard queries in milliseconds
Materialized View	mv_category_by_region	CREATE MATERIALIZED VIEW on fct_daily_sales	Category performance across regions — auto-refreshes on base table change
Clone — dev env	globalmart_dev_db	CREATE DATABASE ... CLONE globalmart_db	Instant production-scale dev environment. Zero cost until modified.
Clone — pre-migration	fct_daily_sales_backup	CREATE TABLE ... CLONE fct_daily_sales	Safe rollback point. Rename swap for zero-downtime rollback.
Clone — audit snapshot	fct_daily_sales_q4_2023	CREATE TABLE ... CLONE ... AT(TIMESTAMP)	Q4 2023 state frozen for auditors. Zero cost.
Clone — A/B test	fct_daily_sales_test_logic	CLONE then ALTER to add new column	Test new logic on clone — production table untouched

6. Feature Reference — What Each Feature Does and Why

This section explains every feature in the context of the GlobalMart project — what it does, why it is used here, and what output proves it is working correctly.

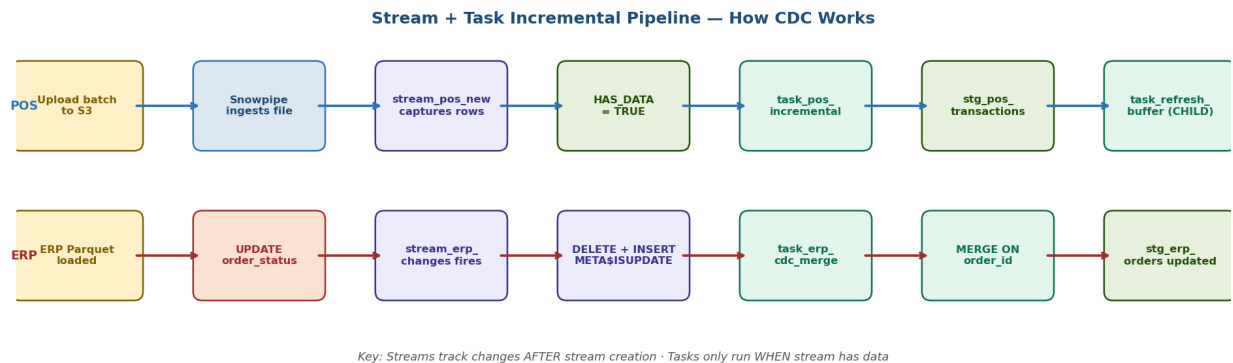


Figure 5: Stream + Task CDC pipeline — both POS (APPEND_ONLY) and ERP (Standard) pathways shown.

6.1 COPY INTO — Three Formats, One Command

COPY INTO is Snowflake's bulk loader. The same command loads all three file formats — only the file format object and column mapping change. Key properties: idempotent (Snowflake tracks loaded files and skips duplicates), parallel (loads multiple files simultaneously), and auditable (every run recorded in COPY_HISTORY).

Why three formats exist in this project
CSV: POS system exports daily flat files. 19 columns, positional mapping (\$1 through \$19).
Parquet: ERP system produces columnar files. Named field extraction via \$1:field_name::TYPE.
JSON: IoT devices emit nested event documents. \$1 stores full document as VARIANT for later flattening.
In real projects you rarely choose the format — the source system dictates it. Handling all three is the skill.

Key SQL — COPY INTO JSON into VARIANT column:

```
COPY INTO raw.iot_events_raw (event_id, event_type, store_id, event_ts, raw_payload,
_source_file)
FROM (
    SELECT $1:event_id::VARCHAR, $1:event_type::VARCHAR, $1:store_id::VARCHAR,
           $1:timestamp::TIMESTAMP_NTZ,
           $1,
           -- $1 alone = entire JSON object into VARIANT
           METADATA$FILENAME
    FROM @global_mart_db.integrations.stg_iot
)
FILE_FORMAT = (FORMAT_NAME = 'global_mart_db.integrations.ff_json')
```

```
ON_ERROR      = 'CONTINUE';
```

6.2 LATERAL FLATTEN — Exploding Nested Arrays

LATERAL FLATTEN converts one row with an array into N rows — one row per array element. Without it, the readings[] and alerts[] arrays inside the IoT JSON are inaccessible as queryable columns.

Three levels of flattening in this project
Level 1: LATERAL FLATTEN(input => raw_payload:readings) → 1 event becomes 2 sensor rows
Level 2: LATERAL FLATTEN(input => raw_payload:alerts) WHERE ARRAY_SIZE > 0 → alert events only
Level 3: Both arrays flattened in one query (multi-level flatten) for correlated sensor-alert analysis
Verification: v_iot_sensor_readings COUNT must be exactly 2× iot_events_raw COUNT.
If counts are equal: LATERAL keyword is missing. If less than 2×: some events have only 1 reading.

Key SQL — LATERAL FLATTEN on readings array:

```
SELECT e.event_id, e.store_id, e.event_ts,
       e.raw_payload:metadata:firmware::VARCHAR AS firmware,
       e.raw_payload:metadata:battery_pct::INTEGER AS battery_pct,
       r.value:sensor::VARCHAR AS sensor_name,
       r.value:value::FLOAT AS sensor_value,
       r.value:unit::VARCHAR AS sensor_unit,
       r.index AS reading_seq
FROM global_mart_db.raw.iot_events_raw e,
     LATERAL FLATTEN(input => e.raw_payload:readings) r;
```

6.3 Snowpipe — Auto-Ingest for IoT Batches 03–05

Snowpipe eliminates the manual COPY INTO step. A pipe object wraps a COPY INTO statement and Snowflake monitors the stage for new files, triggering the copy automatically when they arrive.

- The pipe object is defined once — it is just a named COPY INTO statement.
- AUTO_INGEST=TRUE requires an S3 external stage (storage integration mandatory) and SQS event notification setup.
- Idempotency proof: load iot_events_batch_03.json twice — second run shows STATUS=SKIPPED in COPY_HISTORY.
- Monitor with: SELECT SYSTEM\$PIPE_STATUS('raw.pipe_iot_events') — shows pendingFileCount and lastIngestedTimestamp.

Key SQL — pipe definition:

```
CREATE OR REPLACE PIPE raw.pipe_iot_events
  AUTO_INGEST = TRUE
  COMMENT = 'Auto-ingest pipe for IoT JSON batches 03-05'
AS
```

```

COPY INTO raw.iot_events_raw (event_id, event_type, store_id, store_name, event_ts,
device_id, raw_payload, _source_file)
FROM (SELECT $1:event_id::VARCHAR, $1:event_type::VARCHAR, $1:store_id::VARCHAR,
$1:store_name::VARCHAR, $1:timestamp::TIMESTAMP_NTZ, $1:device_id::VARCHAR,
$1, METADATA$FILENAME FROM @global_mart_db.integrations.stg_iot)
FILE_FORMAT = (FORMAT_NAME = 'global_mart_db.integrations.ff_json');

```

6.4 Time Travel — Three Methods

Time Travel lets you query the historical state of a table within a configurable retention window. Three different methods cover three different operational recovery scenarios.

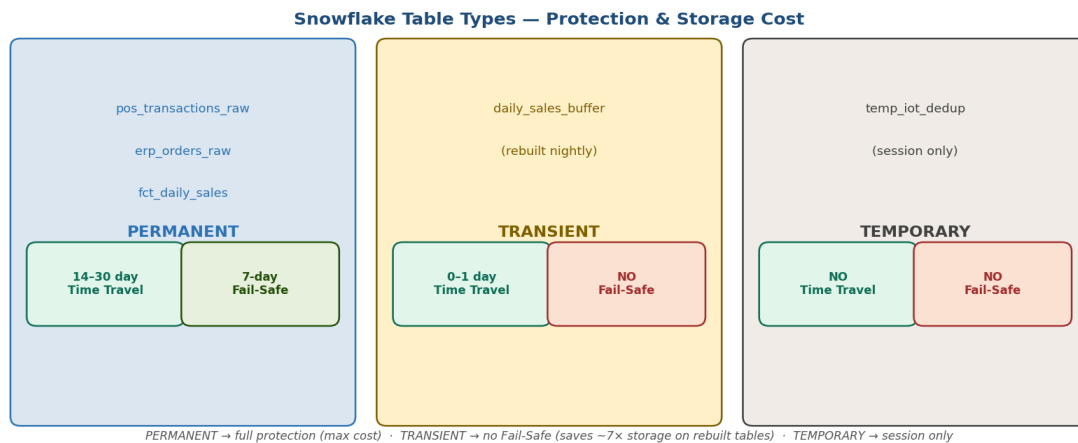


Figure 6: Table types and their Time Travel and Fail-Safe protection levels.

Method	Syntax	Scenario in This Project	Recovery Action
AT(OFFSET)	AT (OFFSET => -N seconds)	Compare POS table before and after May-Aug batch loaded	Row count 5 minutes ago vs now — difference = batch size
AT(TIMESTAMP)	AT (TIMESTAMP => '2023-01-15 18:00:00'::TIMESTAMP_NTZ)	Finance wants revenue at exact first-load timestamp	Query returns Jan-Apr only — shows pre-pipeline state
BEFORE(STATEMENT)	BEFORE (STATEMENT => 'query_id')	Analyst ran UPDATE setting discount_pct=-99 for Clothing/Marmara	MERGE to restore correct values from pre-UPDATE state
UNDROP TABLE	UNDROP TABLE schema.table_name	erp_inventory_raw accidentally dropped	Instant restore — COUNT(*) = 160 rows recovered

6.5 Fail-Safe

Fail-Safe is a 7-day disaster recovery window that activates after Time Travel retention expires. It is not self-service — only Snowflake Support can access it. Its primary relevance in this project is cost: by using TRANSIENT tables for objects rebuilt daily, the Fail-Safe storage cost on those tables is eliminated.

- Permanent tables accumulate Fail-Safe bytes over time — visible in INFORMATION_SCHEMA.TABLE_STORAGE_METRICS.
- TRANSIENT tables: FAILSAFE_BYTES is always 0 — no Fail-Safe, max 1-day Time Travel.
- TEMPORARY tables: no Fail-Safe, no Time Travel, session-only.
- Key query: SELECT TABLE_NAME, TABLE_TYPE, FAILSAFE_BYTES FROM INFORMATION_SCHEMA.TABLE_STORAGE_METRICS
- Expected: pos_transactions_raw (PERMANENT) → FAILSAFE_BYTES > 0, daily_sales_buffer (TRANSIENT) → 0